

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	ch. Hemanth sai	Reg. No.:	192225079
Section / Batch:	SLOT A/2022	Date:	13/07/2026

Code of Conduct

I Ch.Hemanth sai(192225079) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Procedure:

- Import the re module for Regular Expression operations.
- Create sample resume data in text format.
- Extract the candidate's name, email, and mobile number using Regular Expressions.
- Detect technical skills such as Python, Java, SQL, Machine Learning, and NLP.
- Extract the candidate's years of experience.
- Generate and display a structured summary of each candidate.
- Display candidates with at least 2 years of experience and Python skill.

CODE:

```
import re
```

```
# Sample resumes
```

```
resumes = [
```

```
"""
```

```
Name: HEMANTH
```

```
Email: hemanth666@gmail.com
```

```
Phone: +91-9876543210
```

```
Skills: Python, SQL, Machine Learning, NLP
```

```
Experience: 3 years
```

```
""",
```

```
"""
```

```
Name: Rahul Sharma
```

```
Email: rahul.sharma@gmail.com
```

```
Phone: 9876501234
```

```
Skills: Java, SQL
```

```
Experience: 1 year
```

```
""",
```

```
"""
```

Name: Priya Reddy
Email: priya.reddy@gmail.com
Phone: +91 9123456789
Skills: Python, Java, NLP
Experience: 5 years
""
]

Function to extract resume information

def extract_resume_info(resume):

Extract Name

name = re.search(r"Name:\s*(.+)", resume)

name = name.group(1).strip() if name else "Not Found"

Extract Email

email = re.search(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume)

email = email.group() if email else "Not Found"

Extract Phone Number

phone = re.search(r"(\+91[-])?\d{10}", resume)

phone = phone.group() if phone else "Not Found"

Detect Skills

skill_pattern = r"Python|Java|SQL|Machine Learning|NLP"

skills = re.findall(skill_pattern, resume, re.IGNORECASE)

Remove duplicates while preserving order

skills = list(dict.fromkeys([skill.title() for skill in skills]))

Extract Experience

exp = re.search(r"(\d+)\s+year", resume, re.IGNORECASE)

experience = int(exp.group(1)) if exp else 0

return {

 "Name": name,

 "Email": email,

 "Phone": phone,

 "Skills": skills,

 "Experience": experience

}

```

# Extract information for all resumes
candidate_profiles = []

print("===== Candidate Profiles =====\n")

for resume in resumes:
    profile = extract_resume_info(resume)
    candidate_profiles.append(profile)

    print("Name      :", profile["Name"])
    print("Email      :", profile["Email"])
    print("Phone      :", profile["Phone"])
    print("Skills     :", ", ".join(profile["Skills"]))
    print("Experience :", profile["Experience"], "years")
    print("-" * 40)

# Filter eligible candidates
print("\n===== Eligible Candidates =====\n")

for candidate in candidate_profiles:
    if candidate["Experience"] >= 2 and "Python" in candidate["Skills"]:
        print("Name :", candidate["Name"])
        print("Experience :", candidate["Experience"], "years")
        print("Skills :", ", ".join(candidate["Skills"]))
        print("-" * 40)

```

OUT PUT :

```
===== Candidate Profiles =====
```

```

Name      : HEMANTH
Email      : hemanth666@gmail.com
Phone      : +91-9876543210
Skills     : Python, Sql, Machine Learning, Nlp
Experience : 3 years

```

```

-----
Name      : Rahul Sharma
Email      : rahul.sharma@gmail.com
Phone      : 9876501234
Skills     : Java, Sql

```

Experience : 1 years

Name : Priya Reddy

Email : priya.reddy@gmail.com

Phone : +91 9123456789

Skills : Python, Java, Nlp

Experience : 5 years

===== Eligible Candidates =====

Name : HEMANTH2

Experience : 3 years

Skills : Python, Sql, Machine Learning, Nlp

Name : Priya Reddy

Experience : 5 years

Skills : Python, Java, Nlp

RESULT :

The program successfully extracts resume details using Regular Expressions and checks whether the candidate is eligible based on experience and Python skill.

Section 2: Regular Expressions – Product Search System

1. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search.

AIM:

To implement a product search system using Regular Expressions for searching products with different search patterns.

Procedure:

-
- Import the re module.
 - Create a list of product names.
 - Accept a search keyword.
 - Perform exact, prefix, suffix, partial, and case-insensitive searches.
 - Display the matching product names.
 - Display the total number of matching products.
-

CODE :

```
import re

products = [

    "Python Book",

    "Java Book",

    "SQL Guide",

    "Python Basics",

    "Machine Learning",

    "NLP Toolkit",

    "Python Notebook"

]

keyword = "Python"

exact = [p for p in products if re.fullmatch(keyword, p, re.IGNORECASE)]
prefix = [p for p in products if re.match(keyword, p, re.IGNORECASE)]
suffix = [p for p in products if re.search(keyword + r"$", p, re.IGNORECASE)]
partial = [p for p in products if re.search(keyword, p, re.IGNORECASE)]

print("Exact Search:", exact)

print("Prefix Search:", prefix)
```

```
print("Suffix Search:", suffix)
print("Partial Search:", partial)
print("\nReport")
print("Exact Matches:", len(exact))
print("Prefix Matches:", len(prefix))
print("Suffix Matches:", len(suffix))
print("Partial Matches:", len(partial))
```

OUTPUT :

Exact Search: []

Prefix Search: ['Python Book', 'Python Basics', 'Python Notebook']

Suffix Search: []

Partial Search: ['Python Book', 'Python Basics', 'Python Notebook']

Report

Exact Matches: 0

Prefix Matches: 3

Suffix Matches: 0

Partial Matches: 3

Result:

The program successfully performs exact, prefix, suffix, partial, and case-insensitive product searches using Regular Expressions and displays the matching products with the total number of matches.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.

Validate the institutional email address.

Validate the course code.

Validate the semester information.

Validate the student's mobile number.

Display appropriate validation messages for each field.

Generate a final registration status report indicating whether the registration is successful.

AIM :

To design and implement a Python-based student registration validation system using Regular Expressions (Regex) to verify the following details before course enrolment

Procedure:

-
- Start the Python program and import the re (Regular Expressions) module.
 - Accept the student's Register Number, Institutional Email, Course Code, Semester, and Mobile Number as input.
 - Validate the Register Number using the appropriate regular expression pattern.
 - Validate the Institutional Email, Course Code, Semester, and Mobile Number using their respective regex patterns.
 - Display a validation message (Valid or Invalid) for each input field.
 - Check whether all the entered details are valid and determine the overall registration status.
 - Display the final registration status as "Registration Successful" if all fields are valid; otherwise, display "Registration Failed", and end the program
-

CODE:

```
import re

# Input Details

reg_no = input("Enter Register Number: ")
email = input("Enter Institutional Email: ")
course = input("Enter Course Code: ")
semester = input("Enter Semester: ")
mobile = input("Enter Mobile Number: ")

status = True

# Register Number Validation

if re.fullmatch(r"\d{2}[A-Z]{3}\d{4}", reg_no):
    print("Register Number: Valid")
```

else:

```
print("Register Number: Invalid")
```

```
status = False
```

Institutional Email Validation

```
if re.fullmatch(r"[a-zA-Z0-9._%+-]+@college\.edu", email):
```

```
    print("Institutional Email: Valid")
```

else:

```
    print("Institutional Email: Invalid")
```

```
    status = False
```

Course Code Validation

```
if re.fullmatch(r"[A-Z]{2,4}\d{3}", course):
```

```
    print("Course Code: Valid")
```

else:

```
    print("Course Code: Invalid")
```

```
    status = False
```

Semester Validation

```
if re.fullmatch(r"[1-8]", semester):
```

```
    print("Semester: Valid")
```

else:

```
    print("Semester: Invalid")
```

```
    status = False
```

Mobile Number Validation

```
if re.fullmatch(r"[6-9]\d{9}", mobile):
```

```
    print("Mobile Number: Valid")
```

else:

```
    print("Mobile Number: Invalid")
```

```
status = False
```

```
# Final Status
```

```
print("\n----- Registration Status----- ")
```

```
if status:
```

```
    print("Registration Successful")
```

```
else:
```

```
    print("Registration Failed")
```

OUTPUT:

Enter Register Number: 23CSE1025

Enter Institutional Email: tarun@college.edu

Enter Course Code: CSE301

Enter Semester: 5

Enter Mobile Number: 9876543210

Register Number: Valid

Institutional Email: Valid

Course Code: Valid

Semester: Valid

Mobile Number: Valid

----- Registration Status -----

Registration Successful

RESULT:

The Python program was successfully implemented using Regular Expressions to validate the student register number, institutional email address, course code, semester, and mobile number. It displayed appropriate validation messages for each field and generated the final registration status, indicating whether the registration was Successful or Failed.

➤ Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____ Date: _____	Signature: _____ Date: _____	Signature: _____ Date: _____